

Apprentissage Profond (I-ILIA-029) -- Travaux pratiques

TP 06 : VIT et Transfer Learning pour la classification d'objets dans un chantier ferroviaire

L'objectif de ce TP est de développer un module de vidéo surveillance qui consiste à classer des images représentant les objets (rails de train, train, pelleteuse) et ouvriers dans un chantier ferroviaire (BD vue dans le TP précédent). Ceci permettra de sécuriser les chantiers ferroviaires grâce à la détection et localisation de tous les objets dangereux sur chantier. Plus concrètement, le modèle devra prendre en entrée une image couleur avant de la traiter par un modèle de type « **vision transformer, VIT** » pour identifier le label de l'image parmi 4 classes possibles (Fig. 1). Pour réaliser ce TP, nous vous proposons d'exploiter les architecture exploitant le mécanisme d'attention « **VIT** » ainsi que les architectures hybrides « **ConvNext** » combinant les convolutions avec le mécanisme d'attention. Bien évidemment le principe de Transfer Learning sera utilisé aussi pour ce TP.



Figure 1 : exemples d'images représentant les classes à identifier avec un VIT

Principe théorique des VIT : Les images sont représentées sous forme des vecteurs de dimensions $[3, 224, 224]$ alors que l'entrée du transformeur doit être une matrice 2D. La première étape consiste donc en un pré-traitement qui va diviser l'image en plusieurs morceaux (appelés patches), dans le cas des images de taille 224, l'image sera divisée sur chaque côté en 16 patches de 14×14 pixels comme illustré à la figure 2. Il est à noter que l'article original applique une opération d'aplatissement (Flatten) tandis que l'implémentation utilise une convolution 2D qui est appliquée à chaque morceau.



Figure 2: Illustration du découpage en patches

Ensuite, un vecteur de classe est ajouté aléatoirement au début de la séquence. C'est à cet emplacement que seront stockées les informations d'appartenance à la classe à la fin de l'entraînement. À chaque composant est également ajouté un vecteur de position : celui-ci encode la position du morceau dans la séquence, tandis que le vecteur associé à la classe est généré de manière aléatoire. En concaténant l'ensemble de ces éléments, on obtient la matrice d'entrée du transformeur (Fig. 2)

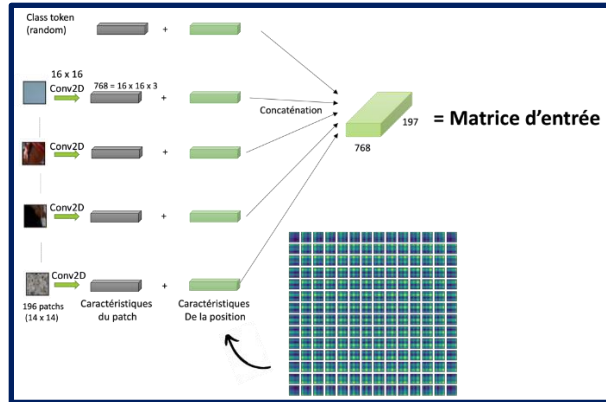


Figure 1 : Etape 1 du ViT « Matrice d'entrée »

A l'intérieur du transformeur, différentes opérations sont appliquées. Celles-ci sont illustrées à la figure 3.

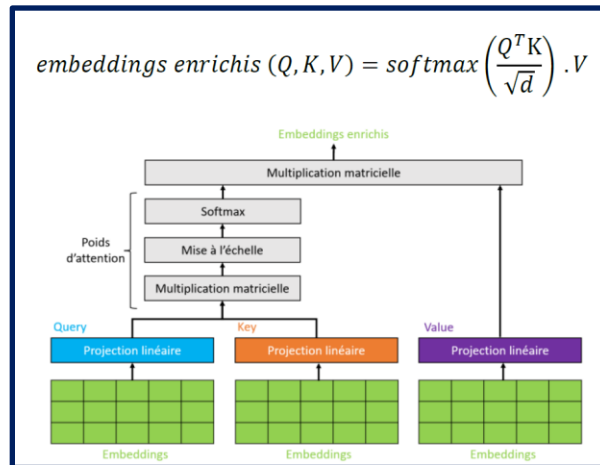


Figure 2 : Etape 2 du ViT « mécanisme d'attention »

En sortie du transformeur, nous avons une matrice de taille 768 x 197 pour une image de 224 x 224 en couleurs. C'est l'entrée de cette matrice (class *Token* initialisé aléatoirement au début) qui va être fournie à la tête de classification qui fournira elle-même un vecteur de probabilité d'appartenance à chaque classe, dans le cas d'ImageNet, 1000 classes.

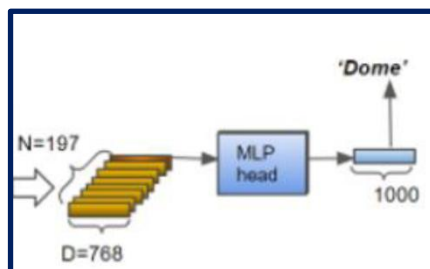


Figure 3 : Etape 3 du ViT « tête de classification »

Enoncé du TP :

- **Question 1 « Vérification du matériel »** : importer le notebook « *TP6_I_ILIA_029_Input_2025.ipynb* » et vérifier que le GPU est bien sélectionné ;
- **Instruction 1 « Installation »** : installer la librairie Pytorch Lightning ;
- **Question 2 :** connecter votre notebook à la plateforme **WanDB** ;
- **Question 3 :** télécharger du jeu de données d'entraînement : **railway-construction-50**
 - **BD railway-construction-50:** <https://nextcloud.ig.umons.ac.be/s/YMGYRp49FH8MRJT/download/railway-construction-50.zip>
- **Instruction 3 : « Importation des libraires »** : importer les libraires nécessaires ;
- **Instruction 4 :** définir les paramètres d'entrainement "Hyper-paramètres" ;
- **Question 4 :** compléter et adapter la classe «Model» vue au TP précédent pour appliquer un transfert d'apprentissage à partir du modèle Vision Transformer «**vit_b_16**» et la fonction d'erreur: **CrossEntropyLoss**

```
self.model = models.vit_b_16(weights="DEFAULT")  
self.model.heads.head = nn.Linear(self.model.heads.head.in_features, num_classes)
```

- **Instruction 5 :** sélectionner une image du dataset, créer une instance de la classe Model.
- **Instruction 6 :** décomposer l'image en patches selon le principe du modèle « **vit_b_16** »
- **Instructions 7 & 8 :** visualiser des patches de l'image
- **Instruction 9 :** créer les caractéristiques de position de chaque patch (position embedding)
- **Instruction 10 :** visualiser les caractéristiques de position
- **Instruction 11 :** combiner les patches avec les caractéristiques de position
- **Instruction 12 :** associer le numéro de la classe à son label

Note : les instructions de 5 à 11 ne **sont pas nécessaires** pour l'entraînement du modèle ViT, elles sont la **uniquement pour expliquer les étapes du fonction du Vit** appelé ci-dessus avec le modèle « `models.vit_b_16` »

- **Instruction 13** : analyser et exécuter la fonction d'affichage des images ;
- **Question 5** : afficher quelques images de chaque classe
- **Question 6** : créer la fonction « *create_data_loader* » de division des données et de création des dataloaders (voir TP5)
- **Question 7** : créer les data loaders (train, val, test) avec la fonction « *create_data_loader* »
- **Question 8** : lancer l'entraînement en utilisant **SGD avec lr='0.01'**
- **Question 9** : afficher vos courbes d'entraînement avec la fonction « *plot_metrics* » (TP 5)
- **Question 10** : vérifier/interpréter les courbes d'entraînement de votre modèle via Wandb ou plot ;
- **Question 11** : évaluer votre modèle avec les données de test
- **Question 12** : exporter le modèle en format torchscript pour la soumission **Kaggle**
- **Question 13** : télécharger la base de données test non labélisée
 - BD de test sans labels: https://nextcloud.ig.umons.ac.be/s/cpm94Z5wYxpzWG/download/test_no_classes.zip
- **Question 14** : normaliser et transformer les données de test
- **Question 15** : générer les prédictions csv à soumettre via Kaggle
- **Question 15** : soumettre vos prédictions au challenge kaggle via ce [lien](#)
- **Question 16** : optimiser les paramètres de votre modèle (et entraîner avec d'autres BD plus grandes) pour améliorer la précision ;
- **Question 17** : refaire l'entraînement et le test avec un autre modèle « **Vit_B_32** ».
- **Question 18** : analyser l'empreinte environnementale du modèle avec notre librairie « **Pytorch_bench** »
- **Question 19** : présenter votre analyse en termes de différences entre les architectures CNN et ViT.
 - BD railway-construction-100 : https://nextcloud.ig.umons.ac.be/s/TaNBtXW8BrES5Te/download/railway-construction_100.zip
 - BD railway-construction-big : <https://nextcloud.ig.umons.ac.be/s/yrrnmPxRc7AKNMs/download/railway-construction-big.zip>

Note : veuillez sauvegarder vos résultats et solutions de ce TP, ils seront exploitables pour la résolution du mini-projet à présenter le **09 mai 2025**.